UNIVERSIDAD TECNOLÓGICA NACIONAL

UTN

Tecnicatura Universitaria en Programación

Metodología de Sistemas II

TRABAJO PRÁCTICO INTEGRADOR

Registro de uso de inteligencia artificial

ETAPA 2 · Análisis, refactoring y verificación del SGSA

Proyecto: Sistema de Gestión de Solicitudes Académicas (SGSA)

Documento de evidencia: prompts, respuestas y capturas de interacción

Contenido: planificación, pruebas, Code Review y UML final

PROMPT

Actuá como analista de sistemas y desarrollador Java, especializado en calidad de software, principios SOLID, Clean Code y técnicas de refactoring.

Estoy desarrollando un Trabajo Práctico Integrador de Metodología de Sistemas II de la UTN, basado en el Sistema de Gestión de Solicitudes Académicas (SGSA), implementado en Java 21.

Necesito elaborar un documento académico en formato Markdown titulado **«Plan de Refactoring — SGSA»**, correspondiente a la Etapa 5 del proyecto.

El documento debe incluir:

- Una introducción breve que explique el objetivo de mejorar la calidad interna del sistema a partir de los code smells identificados y el análisis SOLID y Clean Code.
- La planificación de cuatro refactorings:
- **Extract Method:** dividir el método main() de Main.java.
- **Reubicación de responsabilidades:** separar la lógica de gestión y la presentación por consola de SistemaSolicitudes.java.
- **Introduce Parameter Object:** agrupar los parámetros utilizados para crear una solicitud.
- **Reemplazo de cadenas por Enum:** representar los estados de las solicitudes mediante valores definidos.
- Para cada refactoring, indicar la clase involucrada, el problema detectado, la solución propuesta y el beneficio esperado.
- Una estrategia de implementación progresiva que contemple pruebas antes y después de las modificaciones.
- Las evidencias que se documentarán, incluyendo comparaciones del código, resultados de pruebas y commits en GitHub, conservando la versión original mediante el tag estado-inicial.
- Una conclusión breve sobre los resultados esperados.

Utilizá un lenguaje académico, claro, conciso y adecuado para un estudiante universitario. Organizá el contenido con títulos y subtítulos numerados.

Devolvé únicamente el documento Markdown completo, listo para copiar en docs/plan-refactoring.md. No incluyas código Java ni realices todavía los refactorings.

RESPUESTA

Plan de Refactoring — SGSA

Etapa 5 — Planificación de mejoras

El objetivo de esta etapa es mejorar la calidad interna del Sistema de Gestión de Solicitudes Académicas (SGSA), aplicando cuatro refactorings controlados y documentados.

Las modificaciones se basarán en los code smells identificados y en el análisis de los principios SOLID y Clean Code.

1. Refactorings planificados

Refactoring 1 — Extract Method

- **Clase:** Main.java.
- **Problema:** el método main() concentra múltiples operaciones.
- **Solución:** extraer métodos para organizar las opciones del menú.
- **Beneficio:** mejorar la legibilidad y el mantenimiento.

Refactoring 2 — Reubicación de responsabilidades

- **Clase:** SistemaSolicitudes.java.
- **Problema:** combina lógica de gestión con mensajes por consola.
- **Solución:** trasladar la presentación a una clase específica.
- **Beneficio:** favorecer el principio de responsabilidad única (SRP).

Refactoring 3 — Introduce Parameter Object

- **Clases:** Solicitud.java y componentes de registro.
- **Problema:** el constructor recibe varios datos individuales.
- **Solución:** crear un objeto que agrupe los parámetros necesarios para registrar una solicitud.
- **Beneficio:** simplificar el paso de datos y facilitar futuras modificaciones.

Refactoring 4 — Reemplazo de cadenas por Enum

1. **Clases:** Solicitud.java y SistemaSolicitudes.java.
2. **Problema:** los estados se representan mediante cadenas de texto.
3. **Solución:** definir un enum con los estados permitidos.
4. **Beneficio:** mejorar la consistencia de los datos y reducir errores.

2. Estrategia de implementación

Los cambios se realizarán progresivamente, verificando el funcionamiento después de cada refactoring.

Antes de modificar el código se prepararán pruebas sobre el comportamiento original del sistema. Posteriormente, se ejecutarán nuevamente para detectar posibles regresiones.

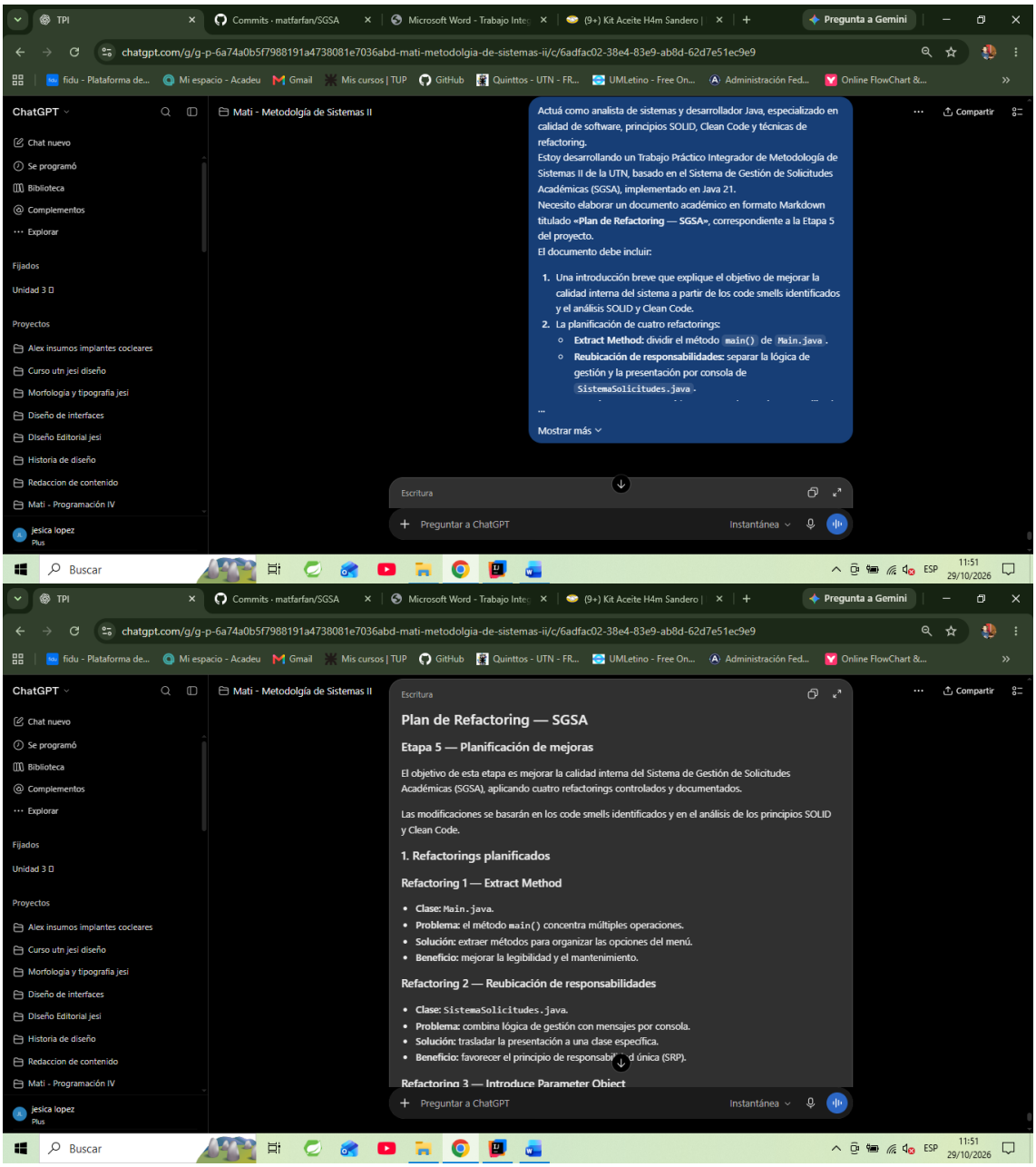
3. Evidencias

Se documentarán los cambios mediante comparaciones del código antes y después, resultados de pruebas y commits en GitHub.

La versión original se conservará mediante el tag estado-inicial.

4. Resultado esperado

Se espera obtener un sistema más organizado, legible y mantenible, conservando las funcionalidades principales de la versión inicial.



PROMT 2

Actuá como analista de sistemas, desarrollador Java 21 y especialista en pruebas de software, JUnit 6, Gradle, SOLID, Clean Code y refactoring.

Estoy desarrollando el Trabajo Práctico Integrador de **Metodología de Sistemas II de la UTN**, basado en el **Sistema de Gestión de Solicitudes Académicas (SGSA)**.

Actualmente estoy en la etapa de preparación de pruebas automatizadas, antes de aplicar los refactorings.

Contexto del proyecto

El SGSA está desarrollado en Java 21, utiliza Gradle 9.3.0 y JUnit 6.0.0, y funciona mediante consola.

El proyecto contiene las siguientes clases:

1. Main.java: menú principal e interacción con el usuario.
2. Estudiante.java: información del estudiante.
3. Responsable.java: información del responsable.
4. Solicitud.java: datos de la solicitud, estado, prioridad y responsable.
5. SistemaSolicitudes.java: registro, búsqueda, listado, filtrado, asignación, cambio de estado y generación de reportes.

Las solicitudes se almacenan en un ArrayList, sin persistencia permanente.

Todavía no se creó la clase de pruebas automatizadas. Necesito definir desde el principio una batería de pruebas que permita evaluar el comportamiento del sistema antes y después de los refactorings.

Mi duda y requisito principal

El TPI exige realizar pruebas **antes y después de los refactorings** para comprobar que las modificaciones no alteraron el comportamiento del sistema.

Por lo tanto, considero que **las pruebas iniciales deben ser exactamente las mismas que se ejecuten después del refactoring**.

No quiero agregar pruebas nuevas únicamente después de modificar el código, porque necesito una comparación válida y documentada entre ambas versiones.

Refactorings planificados

1. **Extract Method:** dividir el método main() de Main.java.
2. **Reubicación de responsabilidades:** separar la lógica de gestión y la presentación por consola de SistemaSolicitudes.java.
3. **Introduce Parameter Object:** agrupar los parámetros utilizados para crear solicitudes.
4. **Reemplazo de cadenas por Enum:** representar los estados mediante valores definidos.

Lo que necesito

1. Analizá si mi criterio de mantener las mismas pruebas antes y después es correcto desde el punto de vista de la ingeniería de software.
2. Definí desde cero una batería completa de pruebas de regresión que cubra las funcionalidades principales del SGSA y los casos negativos relevantes.
3. Incluí registro, búsqueda existente e inexistente, asignación de responsables, cambio de estado, listado, filtrado, reportes y verificación de datos.
4. Explicá paso a paso cómo crear la estructura src/test/java/com/tup/programacion4/ en IntelliJ IDEA.
5. Indicá cómo crear el archivo SistemaSolicitudesTest.java desde cero.
6. Generá el código Java completo de esa clase utilizando JUnit 6, listo para copiar y ejecutar.
7. Diseñá las pruebas para que verifiquen comportamientos observables y no detalles internos de implementación.
8. Tené en cuenta que los métodos actuales de listado, filtrado y reportes utilizan System.out.println(). Si es necesario, capturá la salida de consola para verificarla.

9. Considerá especialmente los cambios previstos con Introduce Parameter Object y Enum. Proponé cómo conservar la compatibilidad de los comportamientos evaluados sin modificar las pruebas originales.
10. No inventes métodos ni constructores que no existan en la versión inicial. Si necesitás confirmar alguna firma, solicitá el código fuente correspondiente antes de generar las pruebas.
11. Diferenciá las pruebas de regresión, que deben mantenerse sin cambios, de posibles pruebas adicionales que podrían incorporarse después para verificar nuevas reglas.
12. Explicá cómo ejecutar las pruebas con Gradle y cómo documentar sus resultados antes y después.

Condiciones obligatorias

1. No modifiques todavía las cinco clases originales del SGSA.
2. No apliques ningún refactoring.
3. No cambies los resultados esperados de las pruebas para conseguir que pasen.
4. No elimines pruebas que fallen sin analizar previamente el motivo.
5. Las pruebas deben ser independientes y reproducibles.
6. La batería inicial debe quedar definida, ejecutada y documentada antes de modificar el código.
7. El código de pruebas debe poder ejecutarse nuevamente después de los cuatro refactorings, manteniendo las mismas verificaciones.
8. Si un cambio de interfaz pública exige compatibilidad, explicá cómo resolverla desde el código refactorizado, sin alterar las pruebas de regresión.

Formato de respuesta

Primero explicá brevemente si mi planteo es correcto.

Después presentá una tabla con todas las pruebas propuestas, indicando el comportamiento y el resultado esperado.

A continuación, explicá **paso a paso cómo crear la carpeta, el paquete y la clase de pruebas en IntelliJ IDEA**.

Finalmente, proporcioná el archivo Java completo, listo para copiar en SistemaSolicitudesTest.java, y los comandos necesarios para ejecutar y documentar las pruebas.

El objetivo es establecer una línea base de regresión confiable para demostrar académicamente que el SGSA mantiene sus funcionalidades después de aplicar los cuatro refactorings.

RESPUESTA

Plan de pruebas automatizadas — SGSA

Etapa 5 — Pruebas de regresión antes y después del refactoring

1. ¿Es correcto utilizar las mismas pruebas antes y después?

Sí. Tu planteo es correcto y es una buena práctica de ingeniería de software.

El propósito de las pruebas de regresión es comprobar que, después de modificar la estructura interna de un programa, los comportamientos existentes continúan funcionando correctamente.

Para el TPI del SGSA vamos a establecer una línea base de pruebas antes de modificar las clases originales.

El procedimiento será:

Versión inicial del SGSA

Código original sin refactorizar

Batería de pruebas JUnit 6

Definir, ejecutar y conservar las pruebas

Aplicar los cuatro refactorings

Mejorar la estructura interna del código

Ejecutar nuevamente las mismas pruebas

Comparar los resultados antes y después

Esto no impide crear pruebas adicionales para nuevas reglas, pero esas pruebas deberán distinguirse de la batería original.

2. Batería definitiva de pruebas de regresión

Propongo 14 pruebas, organizadas por funcionalidad.

N.º	Prueba	Resultado esperado
1	Registrar y buscar solicitud	Recupera la solicitud registrada
2	Buscar solicitud inexistente	Devuelve null
3	Registrar varias solicitudes	Permite recuperar cada una
4	Verificar datos del estudiante	Conserva legajo, nombre y apellido
5	Verificar tipo y prioridad	Conserva los valores registrados
6	Asignar responsable	Actualiza el responsable
7	Asignar responsable a ID inexistente	No genera excepciones ni crea solicitudes
8	Cambiar estado	Actualiza el estado
9	Cambiar estado de ID inexistente	No genera excepciones ni crea solicitudes
10	Listar solicitudes	Muestra las solicitudes registradas
11	Listar sistema vacío	Informa que no existen solicitudes
12	Filtrar solicitudes por estado	Muestra las solicitudes correspondientes
13	Filtrar estado sin coincidencias	Informa que no hay resultados
14	Generar reporte	Muestra los totales correctos por estado

Estas pruebas verifican el comportamiento que ya existe. No vamos a exigir validaciones de identificadores duplicados, prioridades inválidas ni transiciones de estado que el sistema original todavía no implementa.

3. Crear la estructura de pruebas en IntelliJ IDEA

La ubicación correcta es:

```
SGSA/
└── src/
    ├── main/
    │   └── java/
    │       └── com/tup/programacion4/
    │           ├── Main.java
    │           ├── Estudiante.java
    │           ├── Responsable.java
    │           ├── Solicitud.java
    │           └── SistemaSolicitudes.java
    └── test/
        └── java/
            └── com/tup/programacion4/
                └── SistemaSolicitudesTest.java
```

En IntelliJ IDEA:

1. Desplegá src. Si no existe test, hacé clic derecho sobre src → New → Directory y creá test.
2. Dentro de test, creá java si no existe.
3. Hacé clic derecho sobre src/test/java → New → Package.
4. Escribí com.tup.programacion4.
5. Hacé clic derecho sobre ese paquete → New → Java Class.
6. Escribí SistemaSolicitudesTest y presioná Enter.

Si src/test/java no aparece como carpeta de pruebas, hacé clic derecho sobre java → Mark Directory as → Test Sources Root.

Si la estructura ya existe, reutilizala. No es necesario duplicar carpetas ni clases.

4. Código completo de SistemaSolicitudesTest.java

El código siguiente utiliza las firmas originales del SGSA y captura System.out para comprobar las operaciones que imprimen información en consola.

Las pruebas de salida verifican información significativa, sin depender de espacios, saltos de línea o del formato exacto de los mensajes. Esto permite trasladar la presentación a otra clase sin cambiar los resultados esperados.

```
package com.tup.programacion4;


import org.junit.jupiter.api.Test;
```

```
import java.io.ByteArrayOutputStream;
import java.io.PrintStream;
import java.nio.charset.StandardCharsets;

import static org.junit.jupiter.api.Assertions.*;

class SistemaSolicitudesTest {

    private Solicitud crearSolicitud(int id, String estado) {
        Estudiante estudiante =
                new Estudiante(1001, "Ana", "Garcia");

        return new Solicitud(
                id, estudiante, "Certificado",
                "Alta", estado
        );
    }

    private String capturarSalida(Runnable accion) {
        PrintStream salidaOriginal = System.out;
        ByteArrayOutputStream buffer =
                new ByteArrayOutputStream();

        try (PrintStream salida = new PrintStream(
                buffer, true, StandardCharsets.UTF_8)) {
            System.setOut(salida);
            accion.run();
        } finally {
            System.setOut(salidaOriginal);
        }

        return buffer.toString(StandardCharsets.UTF_8);
    }

    @Test
```

```
void registrarYBuscarSolicitudPorId() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();
    Solicitud solicitud = crearSolicitud(101, "Pendiente");

    sistema.registrarSolicitud(solicitud);
    Solicitud resultado = sistema.buscarSolicitudPorId(101);

    assertNotNull(resultado);
    assertEquals(101, resultado.getId());
    assertEquals("Pendiente", resultado.getEstado());
}

@Test
void buscarSolicitudInexistente() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    assertNull(sistema.buscarSolicitudPorId(999));
}

@Test
void registrarVariasSolicitudes() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
    sistema.registrarSolicitud(crearSolicitud(102, "En proceso"));
    sistema.registrarSolicitud(crearSolicitud(103, "Finalizada"));

    assertNotNull(sistema.buscarSolicitudPorId(101));
    assertNotNull(sistema.buscarSolicitudPorId(102));
    assertNotNull(sistema.buscarSolicitudPorId(103));
}

@Test
void verificarDatosDelEstudiante() {
    Solicitud solicitud = crearSolicitud(101, "Pendiente");
```

```
    Estudiante estudiante = solicitud.getEstudiante();

    assertNotNull(estudiante);
    assertEquals(1001, estudiante.getLegajo());
    assertEquals("Ana", estudiante.getNombre());
    assertEquals("Garcia", estudiante.getApellido());
}

@Test
void verificarTipoYPrioridad() {
    Solicitud solicitud = crearSolicitud(101, "Pendiente");

    assertEquals("Certificado", solicitud.getTipo());
    assertEquals("Alta", solicitud.getPrioridad());
}

@Test
void asignarResponsable() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();
    Responsable responsable =
            new Responsable(1, "Maria Lopez", "Secretaria");

    sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
    sistema.asignarResponsable(101, responsable);

    Solicitud resultado = sistema.buscarSolicitudPorId(101);

    assertNotNull(resultado);
    assertNotNull(resultado.getResponsable());
    assertEquals(1, resultado.getResponsable().getId());
    assertEquals("Maria Lopez",
            resultado.getResponsable().getNombre());
}

@Test
```

```
void asignarResponsableASolicitudInexistente() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();
    Responsable responsable =
            new Responsable(2, "Carlos", "Administracion");

    assertDoesNotThrow(() ->
            sistema.asignarResponsable(999, responsable));

    assertNull(sistema.buscarSolicitudPorId(999));
}

@Test
void cambiarEstado() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
    sistema.cambiarEstado(101, "En proceso");

    Solicitud resultado = sistema.buscarSolicitudPorId(101);

    assertNotNull(resultado);
    assertEquals("En proceso", resultado.getEstado());
}

@Test
void cambiarEstadoDeSolicitudInexistente() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    assertDoesNotThrow(() ->
            sistema.cambiarEstado(999, "Finalizada"));

    assertNull(sistema.buscarSolicitudPorId(999));
}

@Test
```

```
void listarSolicitudes() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
    sistema.registrarSolicitud(crearSolicitud(102, "Finalizada"));

    String salida = capturarSalida(sistema::listarSolicitudes);

    assertTrue(salida.contains("101"));
    assertTrue(salida.contains("102"));
    assertTrue(salida.contains("Certificado"));
}

@Test
void listarSistemaVacio() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    String salida = capturarSalida(sistema::listarSolicitudes);

    assertFalse(salida.isBlank());
    assertFalse(salida.contains("101"));
}

@Test
void filtrarSolicitudesPorEstado() {
    SistemaSolicitudes sistema = new SistemaSolicitudes();

    sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
    sistema.registrarSolicitud(crearSolicitud(102, "Finalizada"));

    String salida = capturarSalida(
            () -> sistema.listarPorEstado("Pendiente"));

    assertTrue(salida.contains("101"));
    assertFalse(salida.contains("102"));
```

```
    }

    @Test
    void filtrarEstadoSinCoincidencias() {
        SistemaSolicitudes sistema = new SistemaSolicitudes();

        sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));

        String salida = capturarSalida(
                () -> sistema.listarPorEstado("Finalizada"));

        assertFalse(salida.isBlank());
        assertFalse(salida.contains("101"));
    }

    @Test
    void generarReporte() {
        SistemaSolicitudes sistema = new SistemaSolicitudes();

        sistema.registrarSolicitud(crearSolicitud(101, "Pendiente"));
        sistema.registrarSolicitud(crearSolicitud(102, "Pendiente"));
        sistema.registrarSolicitud(crearSolicitud(103, "En proceso"));
        sistema.registrarSolicitud(crearSolicitud(104, "Finalizada"));

        String salida = capturarSalida(sistema::generarReporte);

        assertTrue(salida.contains("4"));
        assertTrue(salida.contains("2"));
        assertTrue(salida.contains("1"));
        assertTrue(salida.contains("Pendiente"));
        assertTrue(salida.contains("En proceso"));
        assertTrue(salida.contains("Finalizada"));
    }
}
```

Consideraciones técnicas

La batería contiene 14 pruebas, aunque no todas tienen la misma fuerza de verificación. Por ejemplo, las pruebas de listado vacío y filtrado sin coincidencias comprueban condiciones básicas, y la de reporte comprueba la presencia de valores, pero no su asociación exacta con cada categoría.

Para una regresión más estricta, convendría reforzar esas verificaciones con los mensajes exactos de SistemaSolicitudes.java, una vez confirmados. No debemos interpretar que estas 14 pruebas equivalen a cobertura completa.

La captura de consola mediante System.setOut() se utiliza únicamente en pruebas y se restaura con finally. Estas pruebas deben ejecutarse sin paralelismo para evitar interferencias entre capturas.

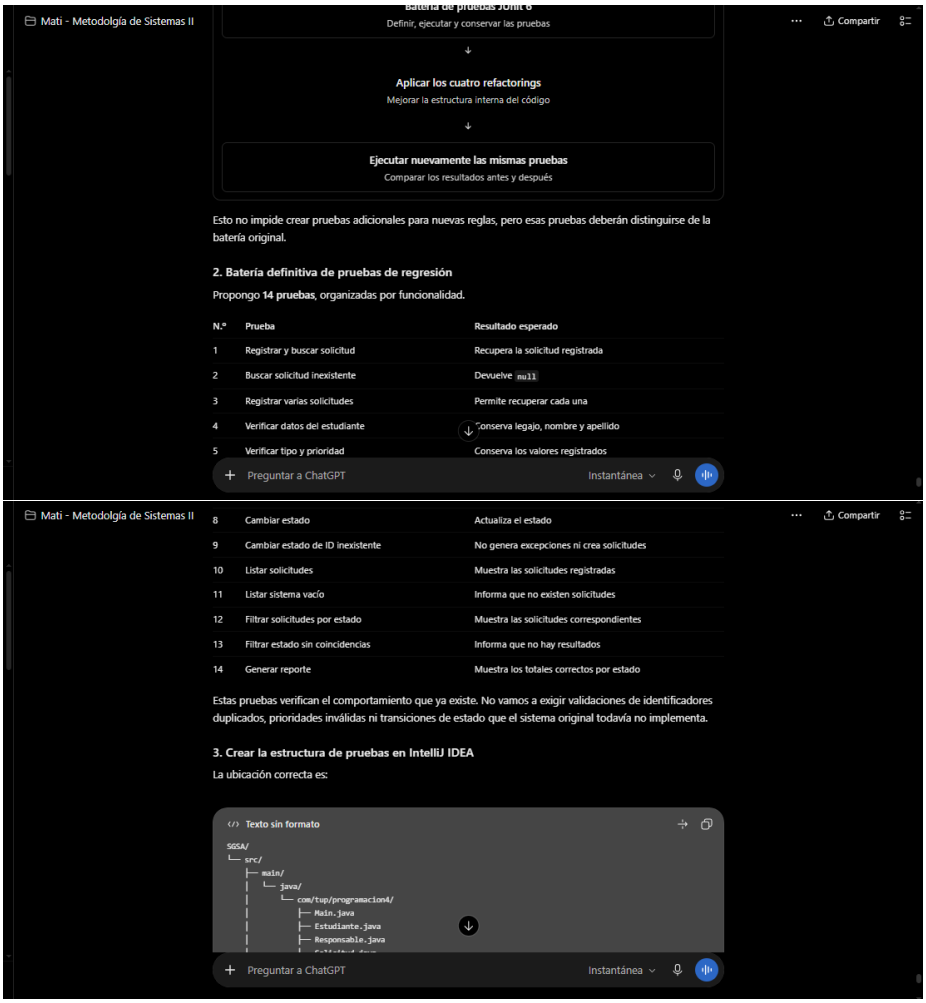
5. Compatibilidad con los cuatro refactorings

Refactoring	Cómo conservar las pruebas
Extract Method	Reorganizar Main.java sin cambiar las operaciones existentes
Reubicación de responsabilidades	Mantener los métodos públicos de SistemaSolicitudes como delegadores, si es necesario
Introduce Parameter Object	Agregar un nuevo constructor y conservar el constructor original como delegador
Reemplazo de cadenas por Enum	Utilizar internamente enum, conservando métodos compatibles que reciban y devuelvan String

Por ejemplo, el sistema refactorizado podrá trabajar internamente con EstadoSolicitud, pero deberá seguir aceptando "Pendiente", "En proceso" y "Finalizada" en las operaciones utilizadas por estas pruebas.

Esto permite cambiar la implementación sin cambiar el contrato público que estamos verificando.


Mati - Metodolgía de Sistemas II
Compartir
Actuá como analista de sistemas, desarrollador Java 21 y especialista en pruebas de software, JUnit 6, Gradle, SOLID, Clean Code y refactoring.
Estoy desarrollando el Trabajo Práctico Integrador de Metodología de Sistemas II de la UTN, basado en el Sistema de Gestión de Solicitudes Académicas (SGSA).
Actualmente estoy en la etapa de preparación de pruebas automatizadas, antes de aplicar los refactorings.
Contexto del proyecto
El SGSA está desarrollado en Java 21, utiliza Gradle 9.3.0 y JUnit 6.0.0, y funciona mediante consola.
El proyecto contiene las siguientes clases:
Main.java : menú principal e interacción con el usuario.
Estudiante.java : información del estudiante.
Responsable.java : información del responsable.
Solicitud.java : datos de la solicitud, estado, prioridad y responsable.
SistemaSolicitudes.java : registro, búsqueda, listado, filtrado,
...
Mostrar más
Plan de pruebas automatizadas — SGSA
Preguntar a ChatGPT
Instantánea
Mati - Metodolgía de Sistemas II
Compartir
Plan de pruebas automatizadas — SGSA
Etapa 5 — Pruebas de regresión antes y después del refactoring
1. ¿Es correcto utilizar las mismas pruebas antes y después?
Sí. Tu planteo es correcto y es una buena práctica de ingeniería de software.
El propósito de las pruebas de regresión es comprobar que, después de modificar la estructura interna de un programa, los comportamientos existentes continúan funcionando correctamente.
Para el TPI del SGSA vamos a establecer una línea base de pruebas antes de modificar las clases originales.
El procedimiento será:
Versión inicial del SGSA
Código original sin refactorizar
Batería de pruebas JUnit 6
Definir, ejecutar y conservar las pruebas
Aplicar los cuatro refactorings
Mejorar la estruct nterna del código
Preguntar a ChatGPT
Instantánea



PROMPT

Ya te compartí previamente el código completo de las ocho clases Java del SGSA y realizamos su revisión técnica.

Ahora necesito que redactes el documento final **docs/code-review.md**, breve, claro y con lenguaje académico.

Organizalo en cinco secciones:

1. **Objetivo:** revisar la calidad del código después de los cuatro refactorings.
2. **Clases revisadas:** mencionar las ocho clases analizadas.
3. **Resultados:** destacar los cuatro refactorings aplicados y presentar una tabla de oportunidades de mejora con prioridades alta, media y baja.
4. **Verificación:** indicar que las versiones inicial y refactorizada superaron las 14 pruebas de regresión y que se comprobó el rechazo de estados inválidos.
5. **Conclusión:** explicar que mejoró la mantenibilidad sin alterar el comportamiento esperado y que las observaciones pendientes quedan como deuda técnica.

Basate exclusivamente en el código que ya analizamos. No inventes hallazgos ni propongas modificar el código ahora. Considerá que la versión final ya está registrada en GitHub con el tag estado-refactorizado.

Devolveme únicamente el documento completo en Markdown, listo para copiar y pegar, sin explicaciones adicionales.

RESPUESTA

Code Review — SGSA

1. Objetivo

Revisar la calidad del código del Sistema de Gestión de Solicitudes Académicas (SGSA) después de aplicar los cuatro refactorings, identificando las mejoras implementadas y las oportunidades de mantenimiento.

2. Clases revisadas

Se analizaron las ocho clases de producción: Main, SistemaSolicitudes, Solicitud, DatosSolicitud, EstadoSolicitud, PresentadorSolicitudes, Estudiante y Responsable.

3. Resultados de la revisión

Aspectos positivos:

1. **Extract Method:** Main presenta métodos más pequeños y organizados.
2. **Move Method:** PresentadorSolicitudes concentra la visualización detallada de las solicitudes.
3. **Introduce Parameter Object:** DatosSolicitud agrupa los parámetros necesarios para construir una solicitud.
4. **Replace String with Enum:** EstadoSolicitud define y restringe los estados permitidos.
5. Se mantuvo la compatibilidad con las funcionalidades existentes.

Oportunidades de mejora:

Prioridad	Observación	Recomendación
Alta	Entradas numéricas y opciones sin validación completa en Main.	Validar las entradas antes de procesarlas.
Alta	No se controlan identificadores duplicados al registrar solicitudes.	Verificar la unicidad del ID.
Media	SistemaSolicitudes conserva lógica de presentación.	Completar la separación de responsabilidades.
Media	Faltan validaciones de datos nulos, vacíos o inválidos.	Incorporar controles en los constructores.
Baja	Persisten cadenas de estado en reportes y objetos de parámetros.	Centralizar su tratamiento mediante EstadoSolicitud.
Baja	Algunos atributos podrían ser final y existe un error tipográfico en registarSolicitud.	Mejorar la inmutabilidad y los nombres.

4. Verificación

La versión inicial y la refactorizada superaron las **14 pruebas automatizadas de regresión**, sin fallos.

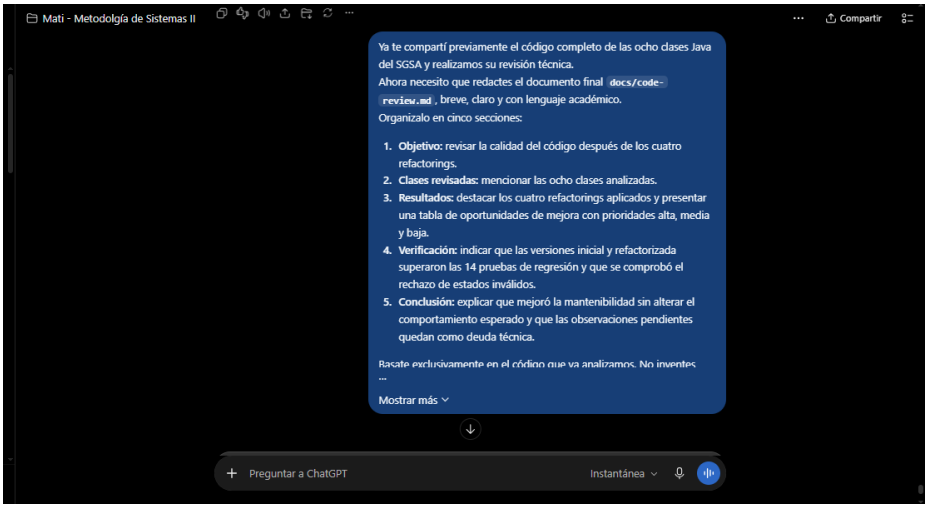
También se comprobó manualmente que el sistema rechaza un estado no válido sin finalizar su ejecución.

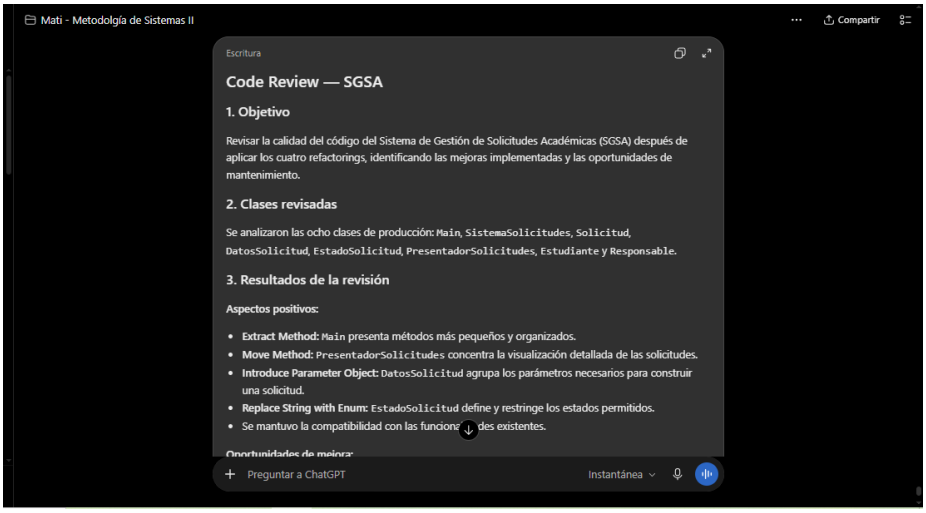
Las evidencias se encuentran en docs/evidencias/ y los resultados están documentados en docs/pruebas-regresion.md.

5. Conclusión

Los cuatro refactorings mejoraron la legibilidad, organización y mantenibilidad del SGSA, sin alterar su comportamiento esperado.

Las observaciones pendientes se registran como deuda técnica para futuras mejoras. No se realizaron nuevas modificaciones durante el Code Review, preservando la versión validada mediante pruebas y registrada en GitHub con el tag estado-refactorizado.





PROMT

Actuá como analista de sistemas y especialista en Java 21, UML, Mermaid, principios SOLID y refactoring.

Estoy desarrollando el Trabajo Práctico Integrador de Metodología de Sistemas II de la UTN, basado en el Sistema de Gestión de Solicitudes Académicas (SGSA).

Necesito generar el documento **SGSA-UML-Final.md**, correspondiente al estado refactorizado del sistema.

Requisitos obligatorios:

1. El documento debe comenzar exactamente con este título:

Diagrama UML de clases — Estado refactorizado del SGSA

2. Debajo del título, incluir exactamente esta introducción:

Este diagrama representa la estructura final del Sistema de Gestión de Solicitudes Académicas (SGSA), después de aplicar los cuatro refactorings.

Se incluyen las ocho clases Java, sus atributos, métodos y relaciones, reflejando las mejoras realizadas en la organización y mantenibilidad del código.

3. A continuación, incluir un único bloque de código Mermaid que comience con classDiagram y direction TB.
4. Representar las ocho clases del sistema:
    1. Main
    2. SistemaSolicitudes
    3. Solicitud
    4. DatosSolicitud
    5. EstadoSolicitud
    6. Estudiante
    7. Responsable
    8. PresentadorSolicitudes
5. Incluir todos los atributos y métodos existentes en el código Java que te proporcionaré, respetando sus nombres, tipos de datos y visibilidad pública o privada.
6. Representar EstadoSolicitud como una enumeración con los valores PENDIENTE, EN_PROCESO y FINALIZADA.
7. Incorporar las relaciones reales entre las clases, utilizando asociaciones, agregaciones y dependencias según corresponda.
8. Mantener el nombre registarSolicitud tal como aparece en el código, sin corregirlo.
9. No inventar atributos, métodos, clases ni relaciones que no estén respaldados por el código proporcionado.
10. No modificar el código Java ni proponer refactorings adicionales.

Formato de salida:

Devolvé únicamente el documento Markdown completo, listo para copiar y pegar en:

docs/evidencias/estado-refactorizado/SGSA-UML-Final.md

El resultado debe contener el título, los dos párrafos introductorios y un único bloque de código delimitado por tres acentos graves con el lenguaje mermaid.

No agregues explicaciones, conclusiones ni texto fuera del documento.

Importante: utilizá exclusivamente el código fuente de las ocho clases Java que te compartí para construir el diagrama. Si falta información, solicitámela antes de generar el resultado.

RESPUESTA

Actuá como analista de sistemas y especialista en Java 21, UML, Mermaid, principios SOLID y refactoring.

Estoy desarrollando el Trabajo Práctico Integrador de Metodología de Sistemas II de la UTN, basado en el Sistema de Gestión de Solicitudes Académicas (SGSA).

Necesito generar el documento **SGSA-UML-Final.md**, correspondiente al estado refactorizado del sistema.

Requisitos obligatorios:

1. El documento debe comenzar exactamente con este título:

Diagrama UML de clases — Estado refactorizado del SGSA

2. Debajo del título, incluir exactamente esta introducción:

Este diagrama representa la estructura final del Sistema de Gestión de Solicitudes Académicas (SGSA), después de aplicar los cuatro refactorings.

Se incluyen las ocho clases Java, sus atributos, métodos y relaciones, reflejando las mejoras realizadas en la organización y mantenibilidad del código.

3. A continuación, incluir un único bloque de código Mermaid que comience con classDiagram y direction TB.
4. Representar las ocho clases del sistema:
    1. Main
    2. SistemaSolicitudes
    3. Solicitud
    4. DatosSolicitud
    5. EstadoSolicitud
    6. Estudiante
    7. Responsable
    8. PresentadorSolicitudes
5. Incluir todos los atributos y métodos existentes en el código Java que te proporcionaré, respetando sus nombres, tipos de datos y visibilidad pública o privada.
6. Representar EstadoSolicitud como una enumeración con los valores PENDIENTE, EN_PROCESO y FINALIZADA.
7. Incorporar las relaciones reales entre las clases, utilizando asociaciones, agregaciones y dependencias según corresponda.
8. Mantener el nombre registarSolicitud tal como aparece en el código, sin corregirlo.
9. No inventar atributos, métodos, clases ni relaciones que no estén respaldados por el código proporcionado.
10. No modificar el código Java ni proponer refactorings adicionales.

Formato de salida:

Devolvé únicamente el documento Markdown completo, listo para copiar y pegar en:

docs/evidencias/estado-refactorizado/SGSA-UML-Final.md

El resultado debe contener el título, los dos párrafos introductorios y un único bloque de código delimitado por tres acentos graves con el lenguaje mermaid.

No agregues explicaciones, conclusiones ni texto fuera del documento.

Importante: utilizá exclusivamente el código fuente de las ocho clases Java que te compartí para construir el diagrama. Si falta información, solicitámela antes de generar el resultado.



